



Formation Kubernetes

Session 11 — CI/CD et Déploiement Multi-Environnement

La session finale : du code à la production

Reflekt Lab | 22h de formation | 11 sessions

Agenda de la session



Recap Session 10

Probes, HPA, PDB, NetworkPolicy, ressources, monitoring



GitHub Actions & Workflows

Déclencheurs, jobs, steps, Workload Identity Federation



Multi-environnement

Staging vs Production : branches, tags, secrets



Démo live

Déploiement en temps réel, pod rolling update



Le cycle CI/CD

Code → Build → Test → Push Image → Deploy → Monitor



Stratégies de déploiement

Rolling Update, Blue-Green, Canary — quand les utiliser



Walkthrough RFLKT

Pipelines réels staging.yml et prod.yml



Clôture & Récap 11 sessions

Votre parcours, takeaways clés, ressources pour aller plus loin

Durée totale : 2h

Recap — Session 10 : Production-Ready



Probes

livenessProbe, readinessProbe, startupProbe —
détection de pods morts



HPA

Horizontal Pod Autoscaler — scale basé sur
CPU/mémoire/custom metrics



PDB

PodDisruptionBudget — évite de tuer tous les pods
pendant un drain



NetworkPolicy

Firewall K8s — restriction du trafic entre
pods/namespaces



Ressources

requests/limits (CPU, mémoire) — scheduling et QoS

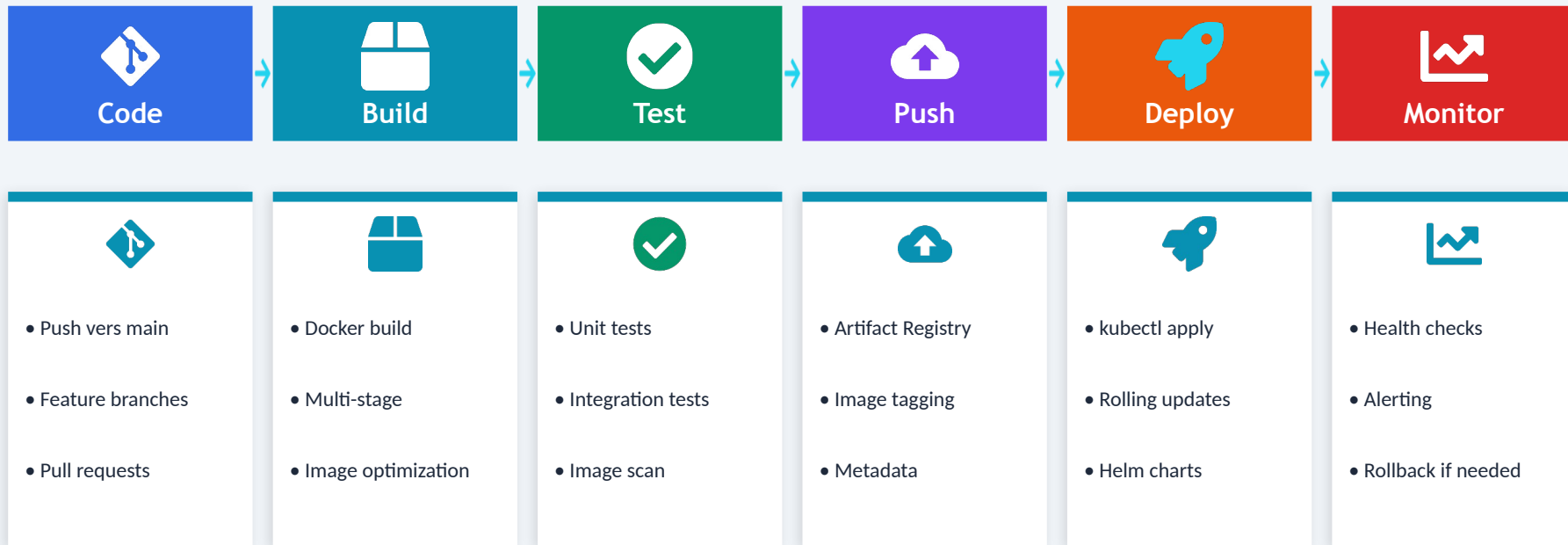


Monitoring

Prometheus, Grafana — observabilité et alerting

Ces concepts rendent une app K8s prête pour la production

Le cycle CI/CD : de la source à la production



Automatisation complète : du commit au pod en production

GitHub Actions – Structure d'un workflow

```
name: Staging CI/CD
on:
  push:
    branches: [main]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Build image
        run: docker build -t api:latest .
      - name: Deploy to K8s
        run: kubectl set image deploy/api
api=api:latest
```

name

Identificateur du workflow

triggers (on:)

push, pull_request, schedule, manual

jobs

Ensemble de tasks parallèles

uses

Actions réutilisables (community)

run

Commandes shell

Auth: Workload Identity Federation (pas de secrets en clair!)

GitHub Actions n'est qu'un outil — le paysage CI/CD

Même principe partout : un pipeline déclaratif déclenché par un événement Git. Le concept est portable, l'outil change.

Jenkins

Le vétéran. Self-hosted, ultra-extensible (plugins), pipelines Groovy. Puissant mais lourd à maintenir.

GitLab CI/CD

.gitlab-ci.yml + runners, intégré au repo GitLab. Repo + CI + registry tout-en-un.

CircleCI / Travis / Drone

SaaS, config YAML, démarrage rapide. Populaires hors écosystème GitHub.

Argo CD / Flux

GitOps pull-based : le cluster se synchronise au repo (cf. slide GitOps).

Tekton

Pipelines natifs Kubernetes (CRDs). Brique de base de nombreuses plateformes.

Ansible

Plutôt config management / provisioning ; parfois utilisé pour orchestrer un déploiement.

Pourquoi GitHub Actions ici ? Le code est déjà sur GitHub → zéro infra à gérer, gratuit, marketplace d'actions. Le pipeline qu'on construit se transpose tel quel ailleurs.

Dockerfile multi-stage : optimiser les images

Single-stage (inefficace)

```
FROM golang:1.22
WORKDIR /app
COPY . .
RUN go build -o app main.go
CMD ["/app"]
```

Image finale: ~850 MB

Contient Go compiler, git, build tools, etc.

✓ Multi-stage (optimisé)

```
# Build stage
FROM golang:1.22 AS builder
WORKDIR /app
COPY . .
RUN go build -o app main.go
# Runtime stage
FROM scratch
COPY --from=builder /app/app .
CMD ["/app"]
```

Image finale: ~15 MB

Contient JUSTE le binaire compilé (scratch=base vide)

Bénéfices du multi-stage : 56× plus petit | Distribution plus rapide | Moins de vulnérabilités (surface d'attaque réduite)

Stratégies de déploiement

Rolling Update (défaut)

v1 → v1/v2 → v2

- Rollout progressif : 1 pod v2, 1 pod v1
- Zéro downtime
- Rollback facile (revert le deployment)
- lent : $N \text{ pods} = N \times \text{readiness probe duration}$

Standard, safe — 95% des cas

Blue-Green

Blue (v1) ↔ Green (v2)

- 2 versions complètes côte à côte
- Basculement instantané (1 clic)
- Double les ressources (coûteux)
- Rollback immédiat si problème

Changements majeurs, tests avant bascule

Canary

v1 (90%) → v2 (10%)

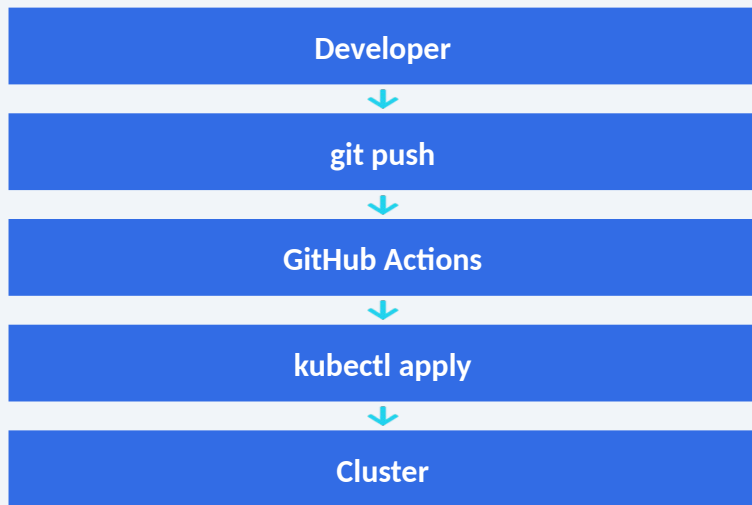
- Graduel : envoi petit % trafic à v2
- Monitor métriques → scale if OK
- Avec Traefik : weighted routing
- Rollback si erreur détectée

Changements critiques, A/B testing

Choisir selon le risque et la tolérance de downtime

GitOps : Source of Truth = Git

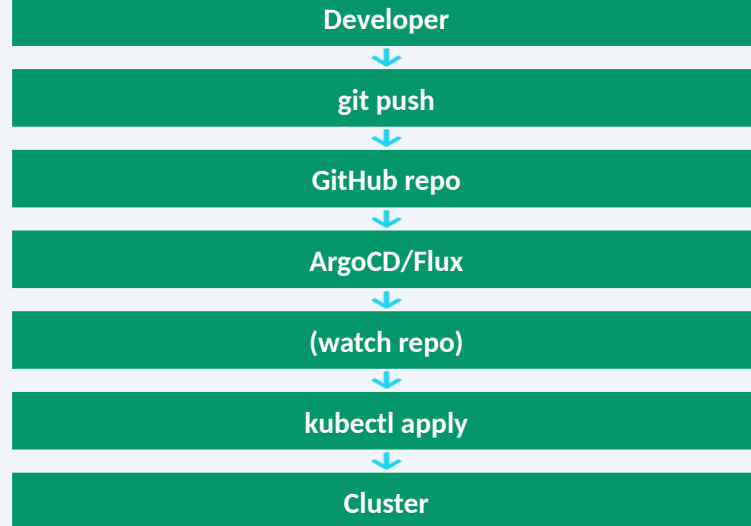
Push-based (GitHub Actions)



Avantages:

- Simple, familier
- Déploiement rapide
- Peu de dépendances

Pull-based (ArgoCD / Flux)

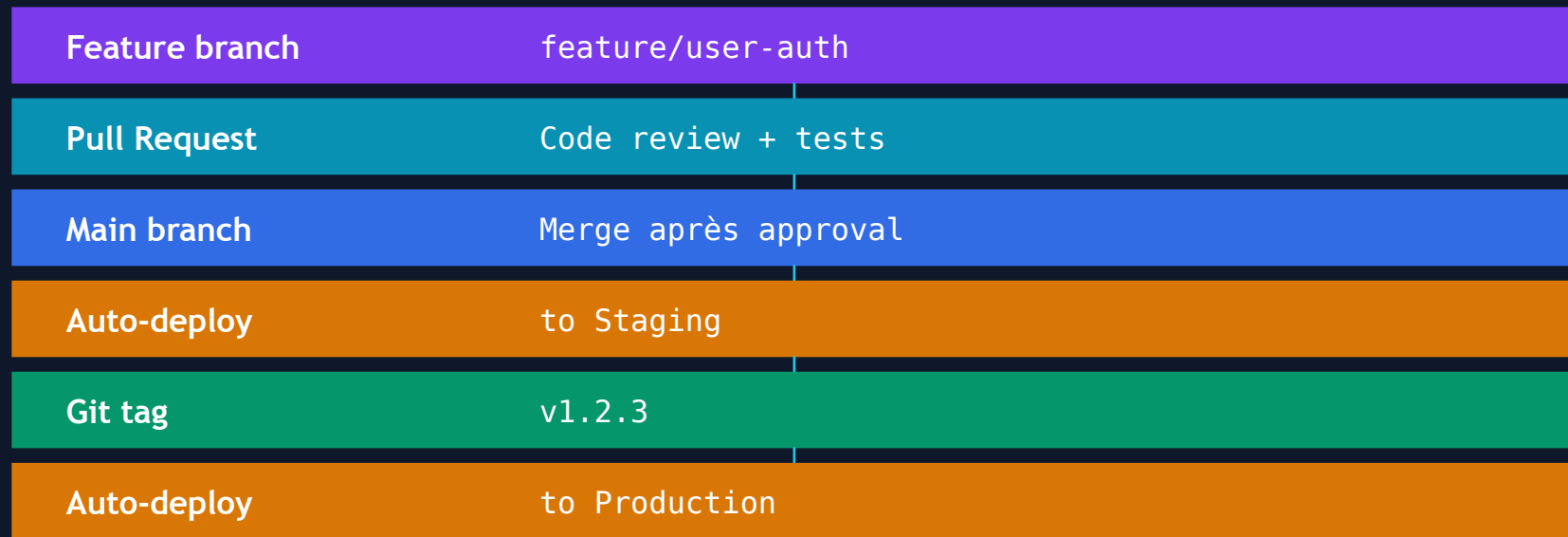


Avantages:

- Audit complet (Git history)
- Drift detection : réalité ≠ Git
- Réconciliation automatique

RFLKT utilise : GitHub Actions (push-based) + Terraform (IaC)

Multi-environnement : Staging et Production



Staging (auto)

Chaque push sur main → déploiement auto. Test rapide.

Production (manual)

Tag créé intentionnellement. Déploiement = release officielle.

Walkthrough : Les pipelines RFLKT réels

Staging Pipeline (.github/workflows/staging.yml)

1. Trigger: push to main
2. Checkout code
3. Setup Gcloud auth (WIF)
4. Terraform init (backend)
5. Terraform plan
6. Terraform apply (CRDs, NEGs)
7. Wait for NEGs
8. Verify backend health
9. Deploy apps to staging GKE

Production Pipeline (.github/workflows/prod.yml)

1. Trigger: release created (git tag)
2. Checkout code at tag
3. Setup Gcloud auth (WIF)
4. Build Docker image
5. Run integration tests
6. Push to Artifact Registry
7. Terraform init (backend prod)
8. Terraform plan
9. Terraform apply (prod infra)
10. Deploy apps to prod GKE
11. Run smoke tests

Key: Terraform gère infra, GitHub Actions gère CI/CD, Workload Identity Federation gère auth

Comparaison : GKE, EKS, AKS

GKE (Google)

- Autopilot : managed nodes
- Release channels : Beta/Regular/Stable
- Control plane gratuit
- Intégration native Anthos

~\$0.10/cluster/h + nodes

Startups, managed experience

EKS (AWS)

- Fargate : serverless pods
- IRSA : fine-grained IAM
- VPC CNI : native AWS networking
- Marketplace integrations

\$0.10/cluster/h + nodes

Enterprise, existing AWS users

AKS (Azure)

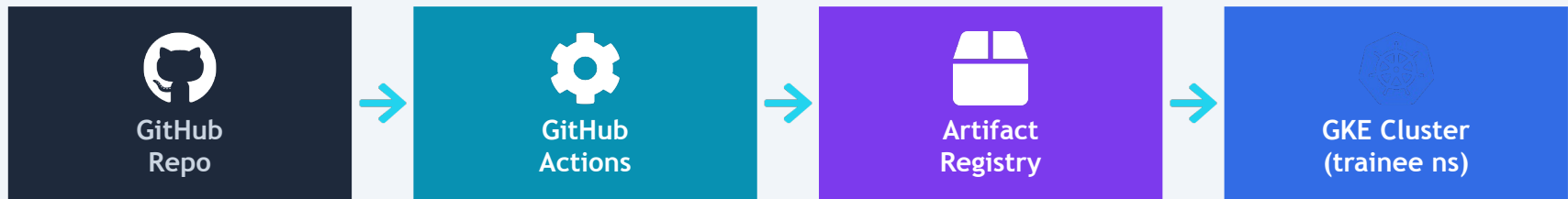
- Azure AD native
- Control plane gratuit
- Spot VMs : cheap compute
- Azure DevOps intégration

Free control plane + nodes

Microsoft shops, cost-sensitive

Bottom line: Kubernetes est Kubernetes — code portable. Choix surtout basé sur coûts + écosystème existant.

Architecture du TP – Déploiement sur GKE



1. git push main

Le développeur pousse son code

2. GitHub Actions

Build de l'image Docker + tests

3. Push to Registry

europa-west9-docker.pkg.dev

4. kubectl set image

Mise à jour de votre namespace trainee

WIF Auth: Service account → GitHub token → GCP credentials



Démo Live — CI/CD en action

1. Montrer le workflow GitHub

Actions tab → Voir les exécutions, logs, status

2. Déclencher un déploiement

git push main ou git tag v1.x.x → Actions se déclenche

3. Suivre le build Docker

Logs : docker build, tests, push to registry

4. Terraform plan & apply

Logs : kubernetes_deployment update, rolling update lancé

5. Observer les pods

kubectl get pods -w → voir les anciens pods terminer, les nouveaux démarrer

6. Vérifier la santé

readinessProbes : pods dans Ready state

7. Tester le service

curl api.example.com → nouveau code en production

Durée : ~10-15 minutes réel, en temps réel



Récap Final – Votre parcours en 11 sessions

S1-S2	S3	S4	S5	S6	S7-S9	S10	S11
Pods, Services, DNS	Ingress, Traefik	Storage, Databases	ConfigMaps, Secrets	StatefulSets	Terraform, GKE	Probes, HPA, PDB	CI/CD (vous êtes ici)

Les 4 grands takeaways

1

Déclaratif > Impératif

Kubernetes = déclarer l'état souhaité (YAML), K8s s'occupe du détail

2

Infrastructure as Code

Terraform : définir infra en code → versionné, auditable, reproductible

3

Automatisation totale

CI/CD : du commit au prod sans clic manuel — zero downtime

4

Résilience par design

Probes, HPA, PDB, monitoring — l'app survit aux pannes

Vous êtes prêts pour la production !